Sets Are Rules, Not Buckets: Intensional Programming in C++23 via `dedekind`*

The `dedekind` Authors

April 24, 2026

Abstract

We present `dedekind` [2], an open-source C++23 library that embeds formal set-theoretic and categorical structures directly into the type system, available under the Apache-2.0 licence. The central design rule is *intensional first, realize it when you mean it*: a set is represented as a predicate $P : A \to \Omega$ over an ambient species A, not as a heap-allocated container of values. This comprehension view, rooted in Lawvere's Elementary Theory of the Category of Sets (ETCS) [3], has two practical consequences that this paper explores:

- Set operations (intersection, complement, union) compose as predicate combinators at compile time, enabling the LLVM backend to perform *structural pruning*—collapsing logically empty intersections into zero-instruction machine code.
- Numeric realization is deferred to explicit boundaries, cleanly separating abstract algebraic reasoning (e.g. over the naturals $\mathbb{N}$) from hardware-constrained IEEE 754 floating-point [4] semantics.

We motivate the design from the practitioner's perspective, beginning with an analyst data-pipeline notebook, and progressively lifting the underlying abstractions to show how recent C++23 advances (concepts, non-type template parameters, named modules) make this approach viable in a mainstream systems language. We identify *Axiomatic Systems Programming* as a nascent paradigm in which the compiler—explicitly *not* a theorem prover, but a structural gate—rules out whole classes of algebraic-law violations at translation time and enables whole classes of structural optimizations that exploit the ruled-out failure modes. As the centrepiece existential proof, we show that a textbook linear-programming instance reduces at compile time to its optimal vertex as a typed constant—the six candidate solves, six feasibility checks, and six objective comparisons all vanish in the emitted IR, which contains only the literal return of the optimum's coordinates. Running the same reduction over the dual-number ring $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ additionally recovers exact sensitivity analysis with no separate derivative pass, recognising forward-mode automatic differentiation as a sibling compile-time reduction in the sense of Elliott [5]. Both reductions share a single mechanism: the feasible polytope is a set-as-predicate, finite vertex candidates enumerate as pairs of binding constraints, and objective-directed argmax collapses the candidate set to a singleton.

Contents

*Draft intended for submission to *Programming* [1], the journal for programming-related research. The paper's scope is deliberately narrower than the companion project report: this document focuses on the sets-as-predicates thesis and compile-time structural pruning; the broader library (categorical foundations, algebra tower, numeric tower, geometry, linear algebra, morphologies, Python facade) is documented in the companion Report distributed alongside the source tree.

1 Introduction

1.1 The Two-Language Problem—From the Analyst's Angle

A data analyst working in Python writes:

```
sales_clean = (
    sales
    .normalize_text(["product_id", "country"], lower=True, strip=True)
    .coerce_numeric(["units", "revenue"], strip_symbols=True)
    .fill_missing(units=0, revenue=0)
    # country is a finite extensional set: ISO 3166-1 alpha-3 codes
    .expect_domain("country", ["DEU", "FRA", "GBR"], on_fail="raise")
    # date is intensional: a point on the real line modelled by ISO 8601
    .expect_iso8601("sale_date")
    # period is a cyclic structure: fiscal quarters repeat annually
    .expect_period("fiscal_quarter", cycle=4)
)

report_plan = monthly_category_report(
    smart_join(sales_clean, products_clean).smart_join(country_meta)
)

summary = report_plan.realize()
```

Listing 1: A declarative analyst pipeline (from the `dedekind` notebook).

Three distinct *kinds* of set structure appear in this single pipeline:

Finite extensional. The call `expect_domain("country", ["DEU", "FRA", "GBR"])` encodes that the `country` column is a member of a known, enumerable set drawn from ISO 3166-1 alpha-3 country codes [6]. The ambient set is finite and fully listed; membership is decidable by equality.

Intensional / infinite. The call `expect_iso8601("sale\_date")` encodes that `sale_date` is a point on a *totally ordered, continuous* ambient structure—the real line $\mathbb{R}$, restricted to the rational subset that ISO 8601 [7] dates inhabit. The ambient set is not enumerated; membership is decided by a parsing predicate. This is the characteristic form of an *intensional* set: defined by a rule rather than a roster.

Cyclic / periodic. The call `expect_period("fiscal\_quarter", cycle=4)` encodes a *cyclic* structure: fiscal quarters are elements of $\mathbb{Z}/4\mathbb{Z}$, a quotient group where quarter 5 wraps back to quarter 1. The ISO 8601 calendar itself exhibits the same periodicity at the week level (52 or 53 ISO weeks per year). Cyclic dimensions are neither purely finite nor purely continuous; their ambient set is a circle, not an interval, and range-based pruning must be replaced by modular arithmetic.

Bootstrapping data quality from guaranteed structural patterns. The three set kinds are not independent. A real-world sales dataset combines all three simultaneously: a country column drawn from a finite ISO 3166-1 roster, a date column inhabiting an intensional time axis, and a period column cycling through fiscal quarters. Their *cross-product* defines a structural skeleton—a lattice of expected cells—against which the actual data can be validated immediately, without any domain-specific business logic.

For example, if a country reports sales data with a known *publication periodicity* (say, Germany publishes monthly, France publishes quarterly), that periodicity is a structural fact expressible as a predicate over the product of the country set and the cyclic period set:

$$\text{Periodicity} = \big\{(c,p) \in \text{ISO3166} \times \mathbb{Z}/n\mathbb{Z} \;\big|\; \text{period\_ok}(c,p)\big\}.$$

A row missing from the expected (c, p) skeleton is a *gap*; a row present outside it is an *anomaly*. Both are detectable as structural violations—set-membership failures—*before* any numeric or business-logic computation is performed.

This turns the three `expect\_*` declarations into a *data quality contract*: the pipeline guarantees, at the boundary of `.realize()`, that every surviving row belongs to the structurally valid region of the joint ambient space. The quality lift reported by the notebook (the delta between the vanilla and smart pivot in the analyst tier) is an empirical measure of how much of the input data falls outside that guaranteed region. Framed this way, data quality improvement is a process of progressively tightening the structural contract—adding more precise predicates—until the residual anomaly rate meets the analyst's tolerance.
The call `.realize()` is an *explicit materialization boundary*: prior to that call, `report_plan` is an intensional description—a recipe without evaluated results. The symmetry between `expect\_*` declarations and `.realize()` is not accidental; it is the central design rule of the `dedekind` library made visible at the Python-facing API tier.

Underneath the Python facade, the library is implemented in C++23. The analyst's pipeline is *not* novel in its high-level form—declarative data-frame APIs are widespread. What is novel is the semantic model that the C++ layer enforces:

1. Sets are *rules* (predicates), not *buckets* (containers).
2. Computation is *intensional until explicitly realized*.
3. The compiler is used as a *verification and optimization engine* for algebraic invariants.

The rest of this paper unpacks these three claims and their consequences for both programmer experience and generated code quality.

1.2 Paper Structure

Section 2 motivates the choice of C++23 as the implementation substrate and introduces the *Axiomatic Systems Programming* paradigm. Section 3 explains the intensional-first design rule and the tension between abstract algebraic numerics and IEEE 754 hardware floating point. Section 4 develops the view of sets as predicates over ambient species and shows how set operations compose symbolically. Section 5 presents compile-time structural pruning, including the open gap for a cardinality-driven no-op intersection. Section 6 surveys related work. Section 7 concludes.

2 Axiomatic Systems Programming

2.1 The Substrate: C++23

The *two-language problem* [8] is the observation that research code is written in an expressive high-level language (Python, R, Julia) while performance-critical execution is delegated to a separately maintained C or C++ backend. Bridging tools such as `nanobind` [9] close the *binary* gap but not the *semantic* gap: the C++ backend cannot reason about high-level mathematical invariants defined in the Python layer.

Recent advances in C++ change this calculus. Three C++23 features are central:

Concepts allow *structural typing*: a type satisfies a concept if it provides the required operations and relations, regardless of its name. This is the compile-time analogue of Haskell type classes, but with zero-cost dispatch and no runtime dictionary.

Non-type template parameters (NTTP) allow lambdas—including predicate functions—to appear as template arguments. A set-builder predicate can therefore be part of the *type*, not merely a runtime value passed at construction.

Named modules enforce a strict directed-acyclic build graph, preventing circular dependencies between algebraic layers and caching verified structural invariants as Binary Module Interface (BMI) files.

Together, these features let the compiler rule out whole classes of algebraic-law violations at translation time and exploit the resulting structural knowledge for optimization. This is *distinct* from theorem-proving: the compiler discharges the proof obligations it is given, it does not search for new ones.

2.2 Structural vs. Nominal Typing: The Juliet Posture

Three distinct schools of structural identification appear in the history of programming languages. Understanding their differences clarifies why C++23 concepts represent a genuine advance.

Nominal typing. Traditional object-oriented frameworks require a type to *declare* its membership. A class is a `Ring` because it inherits from a base class named `Ring`, or implements an interface so named. The *name* is the proof of membership. The compiler checks lineage, not behaviour. A type that provides every ring operation but neglects to inherit the right base class is silently excluded; a type that inherits the right class but provides broken operations is silently accepted.

Duck typing. Dynamic languages (Python, Ruby, early JavaScript) use a runtime structural check colloquially known as *duck typing*: "if it walks like a duck and quacks like a duck, it is a duck." Membership is determined at the call site by testing whether the required method exists and can be invoked. This is genuinely structural—it cares about behaviour, not name—but the check happens *at runtime*, too late to drive compile-time optimization, and errors surface only when a missing method is actually called. Python's `expect_domain` constraint in Listing 1 is duck-typed in this sense: the domain set is a runtime `list`, membership is tested by `in`, and a violation raises an exception at the point of evaluation.

Static structural typing—the Juliet Posture. C++23 concepts combine the structural insight of duck typing with the static, compile-time guarantees of nominal typing—without requiring a declaration of intent. We call this the *Juliet Posture*, after the observation that a rose by any other name would smell as sweet [10]: the library cares about the *scent* (structural invariants), not the *lineage* (declared name).

```
// Nominal: recognised by declared lineage
struct Ring { virtual T operator+(T) const = 0; ... };
struct MyInt : Ring { ... };   // must opt in explicitly

// Duck typing: recognised at runtime by the call site
def generic_sum(a, b):         # Python: fails only when called
    return a + b               # no compile-time guarantee

// Juliet Posture (C++23): recognised statically by behaviour
template <typename T>
concept IsRing = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
    { a * b } -> std::same_as<T>;
    { T::origin() } -> std::same_as<T>; // additive identity
    requires SpeciesTraits<T, std::plus<T>>::is_associative;
};

// int satisfies IsRing without any base class, checked at compile time.
static_assert(IsRing<int>);    // proof obligation discharged at translation
```

Listing 2: The Juliet Posture: structural recognition checked statically, without requiring declared lineage.

The Juliet Posture delivers the best properties of both predecessors: structural sensitivity (behaviour, not name) and static verification (compile time, not runtime). Crucially, because the concept check is a compile-time proof obligation, the compiler can *act on it*: it can specialize code paths, infer additional algebraic properties from the trait registry, and ultimately prune logically empty set expressions to zero machine instructions—none of which is possible with runtime duck typing.

2.3 The Compiler as Structural Gate and Optimization Engine

Two durable uses of a compile-time type system underlie this paradigm, neither of which requires the compiler to be a theorem prover:

1. **Ruling out entire classes of errors.** Encoding algebraic laws as concept requirements makes every instantiation of a generic template an implicit proof obligation discharged at translation time. Failed obligations are type errors, not runtime exceptions. The obligation itself is finite, decidable, and mechanical—the compiler *checks* it, but does not *search for* it.

2. **Enabling entire classes of optimizations.** The LLVM backend exploits the structural knowledge inherited from discharged obligations. When a set is defined by a predicate and two predicates are logically contradictory, the compiler determines at translation time that the resulting intersection is empty and emits no machine instructions for it. This is what we term *structural pruning*, and it is the subject of Section 5.

Both uses are bounded. clang's type checker is not a theorem prover: it has no dependent types, no universal-quantification inference, no proof-search engine. What it does have is `static_assert`, `requires`-clauses, and `constexpr` evaluation—enough to verify a predetermined proof sketch, not enough to generate one. The paradigm claims the reliable half: ruling out, and optimizing under, the classes of structural failures the programmer has named in the type system. The Curry–Howard correspondence between propositions and types is suggestive here, but its full force (dependent products, inductive proofs, universe hierarchies) is not available in C++23; what *is* available is enough to anchor the two uses above.

Bounded yet open-ended

Each use is bounded *individually*, but the paradigm as a whole exhibits a productive open-endedness worth naming. Every class of errors ruled out, and every class of optimizations enabled, tends to expose a next class that the current type machinery does not handle but that the library's structural vocabulary names clearly enough to become a tracked, narrowly-scoped follow-on. In our setting the four axes listed in Section 5.5 are each discovered rather than engineered: they fall out of the halfspace reductions of Section 5 almost by construction.

Penrose offers a related observation about Gödel's incompleteness theorem in *The Road to Reality* [11]: a formal system that closes at one level thereby motivates a next level, not as an exhaustion but as a natural successor. Our setting is more benign than Penrose's philosophical one—the issue tracker plays the role he reserves for the mind's eye, and the successor steps are ordinary software work rather than exits from formal systems—but the pattern is the same. The closest formal analogue in the mathematical-logic literature is Turing's ordinal logics [12], where each stage's consistency feeds the next stage's formation rule. We do not need the full apparatus: the point for a systems-programming paradigm is that closure-at-stage-n is a generative signal for stage-n+1, rather than a terminal achievement.

3 Intensional First, Realize When You Mean It

3.1 The Design Rule

The central methodology of `dedekind` is captured by a single rule:

> *Intensional first, realize it when you mean it.*

The thrust of this rule is to leverage *lazy evaluation*—both at the type level (compile-time symbolic substitution) and at the value level (deferred numeric approximation)—as a principled defence against *Premature Materialization*: the loss of mathematical structure that occurs when a computation is forced to a concrete representation earlier than necessary.

An *intensional* description specifies a mathematical object by its defining properties. An *extensional* representation enumerates its members or computes its value. The analyst's `report_plan` in Listing 1 is intensional; the returned `summary` data frame is extensional. The `.realize()` call is the auditable crossing of that boundary. This framing is close in spirit to intensional database semantics (for example, Majkić's intensional FOL treatment for peer-to-peer data integration [13]), but our engineering contrast with mainstream SQL is explicit: SQL evaluation is typically defined over already-realized relations, while `dedekind` preserves predicate-level structure in the type system and defers materialization to an explicit realization boundary.

3.2 The Tension: Abstract Naturals vs. IEEE Numerics

The motivating tension is the mismatch between the mathematical naturals $\mathbb{N}$ and the machine integers that implement them.

The ring $\mathbb{Z}$ of integers satisfies:

- Associativity of addition: $(a+b)+c = a+(b+c)$.
- Existence of additive inverse: $\forall a, \exists -a$ such that $a + (-a) = 0$.
- No upper or lower bound.

A 32-bit signed `int` satisfies *none* of these exactly: addition overflows, negation of `INT_MIN` is undefined behavior, and the representable range is finite.

IEEE 754 `double` [4] is even further from any abstract field: addition is non-associative due to rounding, and the special values `NaN` and $\pm\infty$ break the law of excluded middle.

`dedekind` encodes these divergences explicitly in the `:species` trait registry:

```
// Ideal integers: associative addition
template <>
struct SpeciesTraits<int, std::plus<int>> {
    static constexpr bool is_associative = true;
    static constexpr bool is_commutative = true;
};

// IEEE double: addition is NOT bit-exact associative
template <>
struct SpeciesTraits<double, std::plus<double>> {
    static constexpr bool is_associative = false; // rounding
    static constexpr bool is_commutative = true;
};

static_assert( SpeciesTraits<int,    std::plus<int>   >::is_associative);
static_assert(!SpeciesTraits<double, std::plus<double>>::is_associative);
```

Listing 3: The species registry records algebraic properties—and their absence.

Generic algorithms that require associativity will fail to compile for `double` unless an explicit numeric policy (e.g. compensated summation, or an interval-arithmetic wrapper) is supplied. The realization boundary is the point at which the programmer chooses which policy to apply.

3.3 Dedekind Cuts as the Canonical Example

The library's namesake—Dedekind cuts—is the canonical illustration. A real number $r \in \mathbb{R}$ is represented as a pair of intensional sets (L, U) where $L = \{q \in \mathbb{Q} \mid q < r\}$ and $U = \{q \in \mathbb{Q} \mid q \geq r\}$. This is entirely symbolic: no floating-point approximation is involved until the programmer requests a concrete rational bound.

```
// Representing e via the series predicate:
//   e = sum_{n=0}^{inf} 1/n!
// The lower cut L_e = { q in Q | q < e }
using LowerCut_e = Set<Rational, [](Rational q) {
    // q is in L_e iff q < partial_sum(1/n!) for sufficient N
    return series_dominates(q, EulerSeries{});
}>;

// At this point no arithmetic has been performed.
// Comparison e < pi is a compile-time structural check on the cuts.
static_assert(cut_less_than<LowerCut_e, LowerCut_pi>);
```

Listing 4: The Euler number e as an intensional lower cut over $\mathbb{Q}$.

The `static_assert` succeeds because the cuts are formally disjoint subsets of $\mathbb{Q}$ by their predicates, not because any floating-point approximation has been compared.

4 Sets Are Rules, Not Buckets

4.1 The Comprehension View

In `dedekind`, a set is not a heap-allocated container of values—a *bucket*—but a *rule*: a predicate $P : A \to \Omega$ over an *ambient species* A, where Ω is a subobject classifier (classical `bool`, Kleene K3, or another registered logic). The set

$$S = \{x \in A \mid P(x)\}$$

exists as a compile-time type. No elements are stored or enumerated until the programmer explicitly requests materialization.

This is the set-builder, or *comprehension*, representation advocated by Lawvere's ETCS framework [3, 14]: set objects are defined by their membership predicates and universal properties, not by extensional storage.

```
// ambient_set<T>(P) creates the intensional set {x in T | P(x)}.
// No values are stored; the predicate IS the set.
auto even_gt_ten = dedekind::category::ambient_set<int>(
    [](int x) { return x % 2 == 0 && x > 10; }
);

// Membership test: O(1), no heap allocation.
bool b = even_gt_ten.contains(12);  // true
bool c = even_gt_ten.contains(7);   // false
```

Listing 5: A set as a compile-time predicate type.

4.2 Set Operations as Predicate Composition

Because sets are predicates, set operations are predicate combinators:

$$\begin{aligned} S_1 \cap S_2 &\triangleq \{x \in A \mid P_1(x) \wedge P_2(x)\} \\ S_1 \cup S_2 &\triangleq \{x \in A \mid P_1(x) \vee P_2(x)\} \\ \overline{S_1} &\triangleq \{x \in A \mid \neg P_1(x)\} \end{aligned}$$

These combinators are composed *entirely at compile time*; the resulting predicate is a new type, not a new runtime object. The compiler can therefore reason about it before any program execution occurs.

```
// Two intensional sets over int:
using Primes = Set<int, [](int n) { return is_prime(n); }>;
using Evens  = Set<int, [](int n) { return n % 2 == 0; }>;

// Intersection: the predicate is P_prime AND P_even
using PrimesAndEvens = Intersection<Primes, Evens>;

// The compiler inspects both predicates.
// Primes > 2 are odd; even primes = {2}.
// With sufficient metadata, this reduces at compile time.
```

Listing 6: Set intersection as type-level predicate conjunction.

4.3 The Subobject Classifier Ω

The ambient logic Ω is a pluggable type parameter. Classical Boolean logic ($\Omega = \{\top, \bot\}$) is the default. For floating-point domains, `dedekind` provides a Kleene K3 logic ($\Omega = \{-1, 0, +1\}$) in which the `NaN` singularity maps to the *Unknown* truth value, cleanly restoring the law of excluded middle: $P \vee \neg P = Unknown$ is not a paradox in K3.

This pluggability is what allows the same set-builder infrastructure to model both exact integer domains (classical logic) and floating-point approximation domains (Kleene logic) without changing the API surface.

5 Compile-Time Structural Pruning

5.1 Structural Pruning in Practice

The payoff of the comprehension view is that the LLVM backend can *prune* a set expression to a no-op when its predicate is statically unsatisfiable.

```
using Above10 = Set<int, [](int x) { return x > 10; }>;
using Below5  = Set<int, [](int x) { return x < 5;  }>;

// Intersection has predicate: x > 10 AND x < 5
// This is logically unsatisfiable for any integer x.
using Impossible = Intersection<Above10, Below5>;

// The compiler verifies emptiness via static_assert.
// The resulting intersection object compiles to zero machine instructions.
static_assert(set_is_empty<Impossible>);
```

Listing 7: Contradictory predicates collapse to zero instructions.

By hoisting the predicate into the type signature, the C++ concept machinery can call `set_is_empty<Impossible>` at compile time. The generated binary contains no membership-test code for `Impossible`: it is structurally pruned by the type checker before code generation.

Showcase witnesses

Two concrete witnesses from the test suite demonstrate this in the actual `dedekind` API.

Showcase 1 — Diagonal contradiction in $\mathbb{R}^2$. On the diagonal $\{(x, y) \mid x = y\}$, the strip predicate $x > 5 \wedge y < 3$ is contradictory (since $x = y > 5$ forces $y > 5$, violating $y < 3$). The intersection is proven empty by `static_assert`; the exported function compiles to a single `ret i1 false`.

```
constexpr auto R2 = R * R;
constexpr auto xy = var<decltype(R2)>;
using R2Point = typename decltype(R2)::Domain;

// Diagonal: { (x, y) ∈ ℝ² | x = y }
constexpr auto diagonal =
    Set{xy % R2 | [](R2Point p) { return p.first == p.second; }};

// Strip: { (x, y) ∈ ℝ² | x > 5 ∧ y < 3 }
constexpr auto strip = Set{
    xy % R2 | [](R2Point p) { return (p.first > 5.0) && (p.second < 3.0); }};

constexpr auto empty_diagonal_cut = diagonal & strip;
using R2Logic = typename decltype(empty_diagonal_cut)::logic_species;

static_assert(empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::False);
```

```cpp
extern "C" __attribute__((noinline)) bool impress_empty_diagonal_cut() {
  return empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::True;
}
```

Listing 8: Showcase 1 source: diagonal contradiction in $\mathbb{R}^2$.

```llvm
define dso_local noundef zeroext i1 @impress_empty_diagonal_cut() #0 {
entry:
  ret i1 false
}
```

Listing 9: Showcase 1 LLVM IR (`clang++-22 -O2`): constant false return.

Showcase 2 — Lattice/square singleton in $\mathbb{C}$. The natural-number lattice (Gaussian integers, $0 \le \mathrm{Re}, \mathrm{Im} \le 3$) intersected with the square $[0.5, 1.5]^2$ contains exactly $c_3 = 1 + i$. The compiler proves all four membership assertions at translation time and folds the exported function to a single `ret i1 true`.

```cpp
constexpr auto c = var<>;

constexpr auto natural_lattice_in_c =
    Set{c % C | [](const Complex<double>& z) {
      return is_integral_coordinate(z.real()) &&
             is_integral_coordinate(z.imag()) &&
             (z.real() >= 0.0) && (z.real() <= 3.0) &&
             (z.imag() >= 0.0) && (z.imag() <= 3.0);
    }};
constexpr auto square_c1_c2 = Set{c % C | [](const Complex<double>& z) {
  return (z.real() >= 0.5) && (z.real() <= 1.5) &&
         (z.imag() >= 0.5) && (z.imag() <= 1.5);
}};
constexpr auto lattice_square_intersection = natural_lattice_in_c & square_c1_c2;
using CLogic = typename decltype(lattice_square_intersection)::logic_species;

static_assert(lattice_square_intersection(Complex<double>{1.0, 1.0}) == CLogic::True);
static_assert(lattice_square_intersection(Complex<double>{0.0, 1.0}) == CLogic::False);

extern "C" __attribute__((noinline)) bool impress_lattice_square_singleton() { /* ... */
```

Listing 10: Showcase 2 source: lattice/square singleton in $\mathbb{C}$.

```llvm
define dso_local noundef zeroext i1 @impress_lattice_square_singleton() #0 {
entry:
  ret i1 true
}
```

Listing 11: Showcase 2 LLVM IR (`clang++-22 -O2`): constant true return.

Showcases 1 and 2 are checked in as LLVM IR fixtures and validated in CI (`make ir-fixture-check`). They exemplify the base case: pair-level predicate composition where constant-folding does the work.

From lambdas to structured predicates: the halfspace DSL

Lambda-based predicates let the optimiser fold at specific call sites, but they erase the *structure* of the constraint. A richer pattern emerges when halfspace bounds live in the *type*: $\{n \in \mathbb{N} \mid n > 5\}$ is represented as `Halfspace<int, 5, Up, Strict>`, with the pivot as a non-type template parameter. Contradiction and cardinality analysis then run in the type system itself, and the reductions are *structural*: no appeal to runtime values is needed.

Showcase 3 — Halfspace contradiction on $\mathbb{N}$. Two opposing halfspaces with non-bridging pivots; the meet reduces to $\varnothing$ by the type-level contradiction rule.

```
constexpr auto n = var <>;

constexpr auto gt_five  = Set{n % N | (n > bound<5>)};
constexpr auto lt_three = Set{n % N | (n < bound<3>)};

// Set-level `&` dispatches through `structured_and` on the halfspace types;
// the contradiction collapses the result's type to ∅<int>.
constexpr ∅<int> empty_meet = gt_five & lt_three;
static_assert(empty_meet == ∅{});

// Computability as a compile-time observable: the parent Sets carry NONE of
// the three tiers; the reduced ∅ carries ALL THREE.
static_assert(!HasDecidableMembership<decltype(gt_five)>);
static_assert(!IsFiniteSet<decltype(gt_five)>);
static_assert(!IsCompileTimeEnumerable<decltype(gt_five)>);
static_assert( HasDecidableMembership<decltype(empty_meet)>);
static_assert( IsFiniteSet<decltype(empty_meet)>);
static_assert( IsCompileTimeEnumerable<decltype(empty_meet)>);
```

Listing 12: Showcase 3 source: halfspace contradiction on $\mathbb{N}$ collapses to $\varnothing$.

```
define noundef zeroext i1 @impress_empty_halfspace_meet() local_unnamed_addr #0 {
  ret i1 false
}
```

Listing 13: Showcase 3 LLVM IR: constant false return.

Showcase 4 — Cardinality-1 reduction on $\mathbb{N}$. The same meet operator, applied to two halfspaces whose bounds pin exactly one integer: the result's type *is* `Singleton<4>`, with the unique inhabitant in the template parameter.

```
constexpr auto n = var <>;

constexpr auto gt_three = Set{n % N | (n > bound<3>)};
constexpr auto lt_five  = Set{n % N | (n < bound<5>)};

// On an integral carrier the meet's cardinality is compile-time-decidable.
// Cardinality 1 elevates the reduction from OrderInterval to Singleton.
constexpr Singleton<4> in_between = gt_three & lt_five;
static_assert(in_between == Singleton<4>{});

// Same computability tightening as Showcase 3, with the element now in the type.
static_assert(!IsCompileTimeEnumerable<decltype(gt_three)>);
static_assert( IsCompileTimeEnumerable<decltype(in_between)>);
```

Listing 14: Showcase 4 source: cardinality-1 meet collapses to $\{4\}$.

```
define noundef zeroext i1 @impress_halfspace_singleton() local_unnamed_addr #0 {
  ret i1 true
}
```

Listing 15: Showcase 4 LLVM IR: constant true return at the unique inhabitant.

The two halfspace showcases make the central claim of the paper mechanically observable: compile-time reduction of an intensional description over a transfinite carrier to a named extensional object (either $\varnothing$ or a `Singleton`) monotonically restores decidability, finiteness, and type-level knowledge of elements. The *reduction boundary* is the *computability boundary*, and the six `static_asserts` flanking each reduction are its mechanical witness.

Further showcases in the source tree exercise the same machinery on $\mathbb{R}$ ambient carriers (showcase 5), non-point intervals with observable cardinality (showcase 6: $-21 < n \leq 21$ on $\mathbb{Z}$, $|\cdot| = 42$), integer lattice intersected with real-valued bounds (showcase 7), and 2D rectangles via structural cartesian product (showcase 8: $42 \times 11 = 462$). All are IR-verified.

5.2 Linear Programming: The Optimum as a Typed Constant

Set-comprehension pruning reduces a compound set *to* a simpler set; the same compile-time machinery, applied under an ordering, reduces a set *to a singleton*. The canonical instance is linear programming: the polytope is an intersection of halfspace predicates (already a set in the comprehension view); the objective is a linear functional; the optimum is the vertex that maximises the objective over the feasible region. When the problem instance is named at the type level, the optimum becomes a typed constant at translation time—the solver is the compiler.

Showcase 9 (centrepiece). Consider a textbook LP:

$$\begin{aligned} \text{maximise} \quad & 3x + 2y \\ \text{subject to} \quad & x + y \leq 4 \\ & 2x + y \leq 6 \\ & x, y \geq 0 \end{aligned}$$

The feasible region is a polygon in $\mathbb{Q}^2$; the optimum is the vertex $(2, 2)$ where the non-axis-aligned constraints $x+y = 4$ and $2x+y = 6$ are simultaneously binding (objective value 10). In `dedekind.optimization:lp`, constraints are NTTPs and the reduction is a variadic template whose output carries the vertex at the type level:

```
using Rat = Rational<long>;
using H1 = Halfspace2D<Rat, Rat{1L}, Rat{1L}, Rat{4L}>;    //  x +  y <= 4
using H2 = Halfspace2D<Rat, Rat{2L}, Rat{1L}, Rat{6L}>;    // 2x +  y <= 6
using H3 = Halfspace2D<Rat, Rat{-1L}, Rat{0L}, Rat{0L}>;   //  x       >= 0
using H4 = Halfspace2D<Rat, Rat{0L}, Rat{-1L}, Rat{0L}>;   //       y  >= 0

using Optimum = decltype(maximize<Rat, Rat{3L}, Rat{2L}, H1, H2, H3, H4>());
static_assert(std::same_as<Optimum, Vec2<Rat, Rat{2L}, Rat{2L}>>);
```

Listing 16: Compile-time LP: the optimum IS a type. From `showcase_09_lp_vertex_typed_constant.cpp`.

The reduction enumerates vertex candidates at compile time (all $\binom{4}{2} = 6$ pairs of binding constraints), solves each 2×2 active-set system via `Invertible2x2`'s closed-form Cramer inverse (exact over $\mathbb{Q}$), filters by feasibility against the non-active constraints, and picks the objective-maximising feasible vertex. Every step is a template specialisation; no runtime iteration survives.

IR witness. The paper's claim is not prose; it is translation-time mechanics. Two `extern "C"` functions in the fixture return the numerator of the optimum's x- and y-coordinates. At `-O2`, `clang`'s optimiser folds the entire LP reduction to literal integers:

```
define noundef i64 @impress_lp_optimum_x() ... {
  ret i64 2
}

define noundef i64 @impress_lp_optimum_y() ... {
  ret i64 2
```

```
}
```
Listing 17: Emitted LLVM IR for Showcase 9. The six candidate solves, six feasibility checks, and six objective comparisons collapse to constant returns.

The IR is checked into the repository as a fixture (`showcase_09_lp_vertex_typed_constant.ll`); a build-time gate (`make ir-fixture-check`) fails CI if optimiser drift ever loses the collapse. The binary contains no LP solver; no simplex tableau; no active-set machinery. The vertex *is* a type, and returning its coordinate *is* a literal.

This closes one of the type-directed-collapse axes listed in Section 5.5 (issue #364) in a worked form: the LP solver is the C++ compiler, for every instance whose size and coefficients are compile-time data.

5.3 Forward-Mode AD as a Sibling Realisation of Compile-Time Derivatives

Running the same LP reduction over the *dual numbers* $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ instead of $\mathbb{Q}$ produces the optimum and its sensitivity to a perturbed parameter as one typed constant—no separate derivative pass, no re-solve of the LP, no finite-difference approximation. The chain rule has already run as the ring arithmetic during the Cramer solve.

We follow Elliott's framing [5]: forward-mode automatic differentiation is the product (f, f') with chain-rule composition, not the "dual number" object *per se*. Dual numbers are one realisation of the $(\mathrm{value}, \mathrm{tangent})$ product in which $\varepsilon^2 = 0$ makes the chain rule a theorem of the ring structure. The `dedekind` library already ships two other realisations of the same abstract derivative: `dedekind.analysis:ftc::derivative_at` (numerical central difference) and `dedekind.algebra:polynomial::RigPolynomial::derive` (exact formal polynomial derivative). `Dual<F>` is the representation that lives at the type level: its primal and tangent are structural fields, so `Dual<Rational<long>>` can appear as `T` in any NTTP-parameterised carrier, including the LP's `Halfspace2D<T, a, b, c>`.

Showcase: parametric LP sensitivity. Perturb H_1's bound by ε—replace $x + y \leq 4$ with $x + y \leq 4 + \varepsilon$. The active set at the optimum is unchanged for infinitesimal perturbation; the Cramer solve now runs over dual numbers:

```
using D = Dual<Rat>;
using H1P = Halfspace2D<D, D{Rat{1L}}, D{Rat{1L}},
                        D{Rat{4L}, Rat{1L}}>;  // bound = 4 + epsilon
using H2D = Halfspace2D<D, D{Rat{2L}}, D{Rat{1L}}, D{Rat{6L}}>;
using H3D = Halfspace2D<D, D{Rat{-1L}}, D{Rat{0L}}, D{Rat{0L}}>;
using H4D = Halfspace2D<D, D{Rat{0L}}, D{Rat{-1L}}, D{Rat{0L}}>;

constexpr auto v = maximize_value<D, D{Rat{3L}}, D{Rat{2L}},
                                  H1P, H2D, H3D, H4D>();
static_assert(v.x.val == Rat{2L}  && v.x.der == Rat{-1L});
static_assert(v.y.val == Rat{2L}  && v.y.der == Rat{2L});
```
Listing 18: Parametric LP over `Dual<Rat>`: optimum + sensitivity as one typed constant. From `lp_test.cpp`.

The primal is the original optimum $(2, 2)$; the tangents $(-1, +2)$ are the partial derivatives of x^* and y^* with respect to the perturbation parameter. A hand derivation confirms: with the active set $\{x + y = 4 + t,\ 2x + y = 6\}$, the solution is $x^*(t) = 2 - t$, $y^*(t) = 2 + 2t$, so the sensitivities are -1 and $+2$. The point of this showcase is not the hand derivation—it is that the library's LP reduction, written once over an unconstrained ring-like carrier, yields exact sensitivity analysis when instantiated over the right ring.

The same `Dual`∘`Rational` layering composes further. Over `Dual<Complex<Rational<long>>>`, the reduction would compute optima and sensitivities over the Gaussian rationals $\mathbb{Q}(i)$; the ring tower is closed under these layerings by construction (every intermediate carrier satisfies the `IsRingLike` concept documented in the companion report). Holomorphic automatic differentiation at compile time is a direct consequence; we leave its application to a follow-up work.

5.4 Algebraic Properties Enable Stronger Pruning

Beyond interval arithmetic, *algebraic* properties of the predicates provide additional pruning opportunities. The `:species` registry records which operations are associative, commutative, idempotent, and so forth. These traits inform the compiler about structural relationships between sets:

- $S \cap \varnothing = \varnothing$ (intersection with the empty set is empty).
- $S \cap \overline{S} = \varnothing$ (intersection with the complement is empty).
- If $S_1 \subseteq S_2$ then $S_1 \cap S_2 = S_1$ (subset absorption).

Each of these equalities can be discharged as a `static_assert` once the relevant algebraic witnesses are in the trait registry, reducing the intersection to the simpler set at compile time.

5.5 From Cardinality Pruning to Type-Directed Collapse

Earlier drafts of this section recorded cardinality-driven pruning (issue #348) as an open contribution target. The halfspace showcases of Section 5 discharge that target: for halfspace-structured predicates over an ordered carrier, the library reduces $S_1 \cap S_2$ to $\varnothing$, to a `Singleton`, or to a typed `OrderInterval` with compile-time-observable cardinality, at translation time and without runtime support.

The resolved form is sharper than the cardinality-count version:

> **Theorem (Type-Directed Collapse, halfspace case).** Let T be an ordered carrier and let P, Q be halfspace predicates over T with pivots carried at the type level as non-type template parameters. The meet $\{x \mid P(x)\} \cap \{x \mid Q(x)\}$ is resolved at compile time to $\varnothing$, a `Singleton<v>`, or an `OrderInterval` whose cardinality (when T is integral) is a compile-time constant. The computability classification of the result is strictly tighter than that of either operand: intensional $\to$ extensional, Ternary $\to$ Classical, opaque $\to$ element-in-type.

This is not just a decision procedure; it is a type-level rewrite rule. The pattern generalizes beyond halfspaces to four independently presentable axes of *type-directed collapse*, each tracked on the issue tracker as follow-on work. For each axis the paper relies on a narrow existential proof (one worked instance + IR fixture + compile-time witness) rather than a completed general framework.

- **Carrier tightening** (issue #362). Intersecting a subset of $\mathbb{N}$ with a subset of $\mathbb{R}$ should land in $\mathbb{N}$: the carrier lattice contributes its own structural reduction via the existing canonical embeddings (`embed__`, `embed_ℚ_ℝ`, …).
- **Smooth optimization** (issue #363). The stationary set $\{x \mid \nabla f(x) = 0\}$ for a quadratic f collapses to a `Singleton<x*>` with the minimizer in the template parameter. Same reduction pattern, applied at the extremum operator.
- **Linear programming** (issue #364, demonstrated in Section 5.2 and Section 5.3). `maximize(c, polytope)` reduces, for compile-time-sized LPs, to a typed vertex via type-level active-set enumeration, and the same reduction over a dual-number carrier yields sensitivity analysis with no extra code path: the solver is the compiler.

- **Lattice-law rewriting** (issue #365). Distributivity and absorption applied at the type level rewrite $(A \cup B) \cap \neg A$ to $B \cap \neg A$ *before* local reduction rules fire, exposing collapses that a purely local analysis would miss.

Each axis sits on the foundation shared with Section 5: NTTP predicates, structured `operator&` dispatch, and the computability tiers visible in the showcases. Taken together they sketch a program of which the current halfspace reductions are the first installment.

6 Related Work

Functional and dependently-typed approaches. Agda [15] and Coq provide dependent type systems in which set-membership proofs are first-class terms. The expressiveness is greater than C++23, but the runtime model differs fundamentally: proof terms are erased at runtime, whereas `dedekind`'s structural pruning targets machine-code generation in a systems-programming context. Haskell [16] supports type-class-based structural reasoning, and libraries such as `algebra` encode ring and field hierarchies. C++ templates differ in that they operate on concrete machine types (with hardware-level algebraic imperfections) rather than erased polymorphic dictionaries.

Exact arithmetic. The GNU Multiple Precision library (GMP) and `mpfr` support arbitrary-precision arithmetic at runtime. `iRRAM` [17] and similar libraries implement computable real arithmetic. `dedekind` is complementary: exact reals (Dedekind cuts) are modelled *intensionally as types*, and realization to a specific arithmetic backend is an explicit programmer choice.

Set-builder type systems. Refinement types [18] attach predicates to types, enabling a form of set-builder representation in ML-family languages. Liquid Haskell [19] extends Haskell with SMT-backed refinement predicates. `dedekind` achieves a subset of this functionality in standard C++23 without a custom type checker, using NTTP-embedded lambdas as the predicate carrier.

Data-pipeline DSLs. Apache Spark, Flink, and the Python `pandas`/`polars` ecosystem provide declarative data pipelines. None of these ground their APIs in formal set theory; domain constraints such as `expect_domain` are runtime assertions, not compile-time type invariants. `dedekind`'s Python facade inherits structural guarantees from the C++ layer while preserving the familiar data-frame programming model.

C++ template metaprogramming libraries. Boost.Hana [20] and `mp11` [21] provide compile-time data structure manipulation. `dedekind` builds on these ideas but targets algebraic *semantic* content (associativity, commutativity, cardinality) rather than purely syntactic structure manipulation.

7 Conclusion

We have presented `dedekind`, a C++23 library that treats sets as *rules over ambient species* rather than containers of values. The intensional-first design rule—defer materialization to explicit boundaries—has two direct payoffs: algebraic reasoning is preserved across the abstract/hardware boundary, and the LLVM backend can prune logically empty set expressions to zero machine instructions.

These ideas are motivated from the bottom up, starting with an analyst data pipeline notebook where `.realize()` is the visible realization boundary, and ascending to the formal set-theoretic and categorical foundations that justify the optimization.

Three themes recur throughout:

1. **Axiomatic Systems Programming.** C++23 concepts, named modules, and NTTP-embedded predicates make it possible to use the compiler—though explicitly not as a theorem prover—to rule out whole classes of algebraic-law violations and to exploit the resulting structural knowledge for optimization, without a separate proof assistant.
2. **Intensional first.** The tension between abstract number theory and IEEE 754 hardware semantics is resolved by encoding the divergence in the `:species` trait registry and deferring arithmetic policy choice to the realization boundary.
3. **Sets are rules.** A set defined by a type-level predicate supports compile-time symbolic manipulation that a runtime container cannot.

The open gap of Section 5.5—cardinality-driven no-op intersection for finite ambient sets—represents the next milestone toward making structural pruning a general-purpose optimization pass available to any `dedekind` user.

References

[1] *<Programming>: The Art, Science, and Engineering of Programming.* `https://programming-journal.org/`. Open-access journal for programming-related research; ISSN 2473-7321.

[2] The dedekind Authors. *`dedekind`: Axiomatic Systems Programming in C++23.* `https://github.com/vincentk/dedekind`. Open-source implementation, Apache-2.0 licensed. 2026.

[3] F. William Lawvere. "An elementary theory of the category of sets". In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[4] IEEE. *IEEE Standard for Floating-Point Arithmetic.* Provides the formal specification for floating-point formats and operations. IEEE. 2019. DOI: `10.1109/IEEESTD.2019.8766229`.

[5] Conal Elliott. "The simple essence of automatic differentiation". In: *Proceedings of the ACM on Programming Languages* 2.ICFP (2018), pp. 1–29.

[6] *ISO 3166-1: Codes for the Representation of Names of Countries and Their Subdivisions — Part 1: Country Codes.* Tech. rep. ISO 3166-1:2020. International Organization for Standardization, 2020. URL: `https://www.iso.org/standard/72482.html`.

[7] *ISO 8601-1: Date and Time — Representations for Information Interchange — Part 1: Basic Rules.* Tech. rep. ISO 8601-1:2019. International Organization for Standardization, 2019. URL: `https://www.iso.org/standard/70907.html`.

[8] Jeff Bezanson et al. "Julia: A Fresh Approach to Numerical Computing". In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: `10.1137/141000671`.

[9] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings.* `https://github.com/wjakob/nanobind`. 2022.

[10] William Shakespeare. *The Most Excellent and Lamentable Tragedy of Romeo and Juliet.* Act 2, Scene 2. John Danter, 1597.

[11] Roger Penrose. *The Road to Reality: A Complete Guide to the Laws of the Universe.* Chapter 16 and surrounding material on Gödel's incompleteness theorem. London: Jonathan Cape, 2004.

[12] Alan M. Turing. "Systems of Logic Based on Ordinals". In: *Proceedings of the London Mathematical Society.* Second Series 45 (1939), pp. 161–228.

[13] Zoran Majkić. “Intensional First-Order Logic for P2P Database Systems”. In: *Journal on Data Semantics* 12 (2009), pp. 47–66. DOI: 10.1007/978-3-642-00684-6_4. URL: https://dblp.org/rec/journals/jods/Majkic09.

[14] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Discusses the construction of the continuum and Dedekind cuts within a categorical framework. Cambridge University Press, 2003.

[15] Ulf Norell. “Towards a practical programming language based on dependent type theory”. PhD thesis. Chalmers University of Technology, 2007. URL: http://www.cse.chalmers.se.

[16] The GHC Development Team. *The Glasgow Haskell Compiler User’s Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html.

[17] Norbert Th. Müller. “The iRRAM: Exact Arithmetic in C++”. In: *Computability and Complexity in Analysis (CCA 2000)*. Vol. 2064. Lecture Notes in Computer Science. Springer, 2001, pp. 222–252.

[18] Tim Freeman and Frank Pfenning. “Refinement Types for ML”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 1991, pp. 268–277.

[19] Niki Vazou et al. “Refinement Types for Haskell”. In: *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2014, pp. 269–282.

[20] Louis-Dionne Chagnon. “Boost.Hana: Your Metaprogramming Nightmare”. In: *CppCon 2015*. https://github.com/boostorg/hana. 2015.

[21] Peter Dimov. *mp11: A C++11 Metaprogramming Library*. https://github.com/boostorg/mp11. 2017.